Flashcard Quizzer: Performance Report

Repository	github.com/dipanjan-acn/ud-ainse-assignment
Commit measured	2e8b2ff (main)
Machine	Apple M5 Pro, 24 GB RAM, macOS 26.7.1
Interpreter	Python 3.11.17 in a virtual environment
Date	2026-10-24

1. Summary

Every measured operation finishes in under 120 ms, including the 100,000-card cases. A 10,000-answer CLI session takes about 110 ms in the latest run. The profile found no single hot spot. The one change tried (stripping each field once) made no measurable difference, so it was reverted. No optimization is included in this commit.

Two limits on these figures.
1. The app rejects deck files over 1 MB, which is about 20,000 short cards. The 100,000-card rows lift that limit inside the benchmark only, so they measure the parser, not a deck the app accepts.
2. Run-to-run variance is large on this machine. CLI startup, which the change did not touch, measured 40 ms in an earlier run and 82 ms in the latest one. Compare changes with an A/B test in one process, not across runs.

2. Latest results

Each operation was repeated 5 times (3 for CLI runs). Times are in milliseconds. The bar is relative to the slowest median.

Operation	Median	Min	Max	Relative median
Load deck of 1,000 cards	0.6	0.6	0.8	
Random session, 1,000 cards, all answered	0.9	0.9	0.9	
Adaptive session, 100 cards, all missed once	0.2	0.2	0.2	
Load deck of 10,000 cards	4.5	4.3	5.5	
Random session, 10,000 cards, all answered	8.8	8.7	9.0	
Adaptive session, 1,000 cards, all missed once	1.4	1.4	1.4	
Load deck of 100,000 cards *(1 MB limit lifted for measurement)*	57.5	57.2	58.3	
Random session, 100,000 cards, all answered *(1 MB limit lifted for measurement)*	111.4	111.0	114.0	
Adaptive session, 10,000 cards, all missed once	13.8	13.8	14.1	
CLI run, 10,000 piped answers, sequential, --stats	112.5	109.8	122.3	
CLI startup (--help)	81.6	80.5	88.8	

3. Optimization investigation

A profile showed the time spread across per-card Python work, with no single hot spot. Two candidates were tested.

Candidate	Before (ms)	After (ms)	Change
10,000 cards	3.2	3.3	+1.6%

100,000 cards	37.3	35.7	-4.1%

- **Strip each text field once (tested, rejected).** The table is an A/B comparison in one process, alternating old and new code. The change is within run-to-run noise, so it was reverted.
- **Stream the deck load (rejected).** The 1 MB limit already bounds memory, and a streaming parser would need a new dependency.
- **Cache the normalized answer per card (not implemented).** The profile suggests about 8% of session time. It would add a cache or a new field to the card model, for a small gain.
- **Keep `deque` for the queue (kept).** Adaptive mode adds missed cards to the back and takes from the front, both in constant time.

4. Method

- **Deck loading:** `load_flashcards()` reads, parses, and validates a generated deck of `{"front", "back"}` cards.
- **Random session:** every card is answered correctly through `run_quiz()`, with output sent to memory rather than a terminal.
- **Adaptive session:** one tenth of the deck is answered wrongly on the first pass, so every card is re-queued and answered correctly on its return.
- **CLI run:** 10,000 answers piped to the app as a separate process with `--stats` and `--no-color`. Each run must exit with code 0 and report 10,000 attempts, or the timing is discarded.
- **Startup:** `python main.py --help`, which includes process start, imports, and argument parsing.

5. Limitations

- One machine and one Python version. Other hardware will give different figures.
- Input was piped and output went to memory. Typing answers in a terminal takes far longer than any figure here.
- No concurrency or memory-usage measurements were taken.
- The benchmark scripts are in the session's scratch folder and are not in the repository.